7.2.1 Introduction

SQL injection attacks aim at influencing database queries by manipulating web application parameters. A popular goal of SQL injection attacks is to bypass authorization. Another goal is to carry out data manipulation or reading arbitrary data. Here is an example of how not to use user input data in a query:

```
Project.where("name = '#{params[:name]}'")
```

This could be in a search action and the user may enter a project's name that they want to find. If a malicious user enters ' OR 1 --, the resulting SQL query will be:

```
SELECT * FROM projects WHERE name = '' OR 1 --'
```

The two dashes start a comment ignoring everything after it. So the query returns all records from the projects table including those blind to the user. This is because the condition is true for all records.

7.2.2 Bypassing Authorization

Usually a web application includes access control. The user enters their login credentials and the web application tries to find the matching record in the users table. The application grants access when it finds a record. However, an attacker may possibly bypass this check with SQL injection. The following shows a typical database query in Rails to find the first record in the users table which matches the login credentials parameters supplied by the user.

```
User.first("login = '#{params[:name]}' AND password =
'#{params[:password]}'")
```

If an attacker enters ' OR '1'='1 as the name, and ' OR '2'>'1 as the password, the resulting SQL query will be:

```
SELECT * FROM users WHERE login = '' OR '1'='1' AND password = '' OR
'2'>'1' LIMIT 1
```

This will simply find the first record in the database, and grants access to this user.

7.2.3 Unauthorized Reading

The UNION statement connects two SQL queries and returns the data in one set. An attacker can use it to read arbitrary data from the database. Let's take the example from above:

```
Project.where("name = '#{params[:name]}'")
```

And now let's inject another query using the UNION statement:

```
') UNION SELECT id,login AS name,password AS description,1,1,1 FROM
users --
```

This will result in the following SQL query:

```
SELECT * FROM projects WHERE (name = '') UNION
  SELECT id,login AS name,password AS description,1,1,1 FROM users --'
```

The result won't be a list of projects (because there is no project with an empty name), but a list of user names and their password. So hopefully you encrypted the passwords in the database! The only problem for the attacker is, that the number of columns has to be the same in both queries. That's why the second query includes a list of ones (1), which will be always the value 1, in order to match the number of columns in the first query.